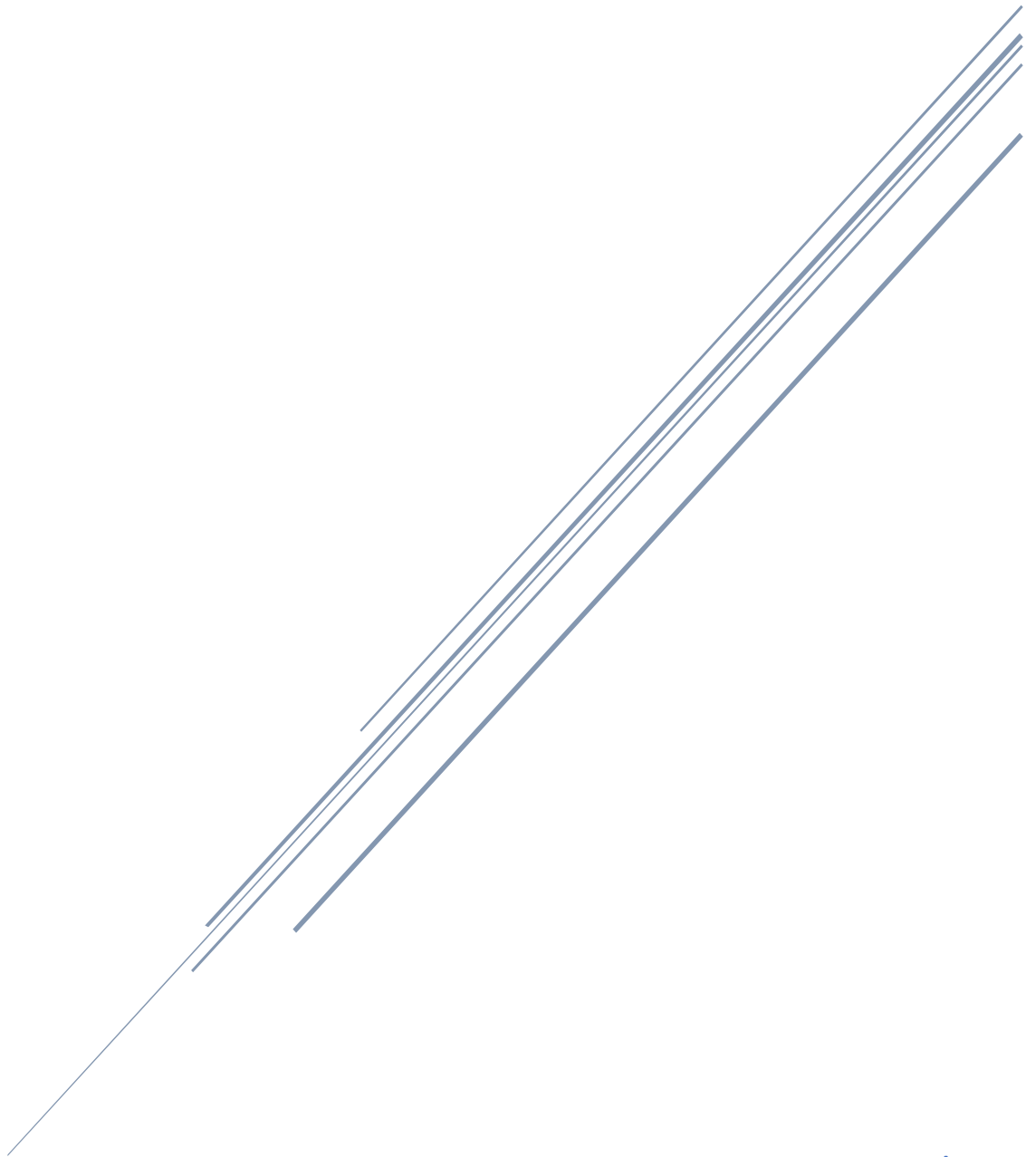


MEMORIA DEL PROYECTO THOTH TASKS

PROYECTO INTERMODULAR



Georgii Sytnik
DAM 2 DIA

1. Introducción	2
1.2 Justificación del proyecto	3
3. Objetivos	4
3.1 Objetivos específicos	5
4. Análisis de necesidades y requisitos	5
4.1 Problema detectado	6
4.2 Usuarios de la aplicación	7
4.3 Alcance del proyecto	7
4.5 Análisis de mercado	8
4.6 Requisitos funcionales	8
4.7 Requisitos no funcionales	10
5. Diseño de la solución software	10
5.2 Arquitectura general	11
5.3 Modelo de datos	11
5.4 Nombre y logo	13
5.5 Diseño de la interfaz y Flujos	14
6. Planificación del proyecto	15
6.1 Metodología empleada	15
6.2 Fases	15
6.3 Cronograma pensada	16
6.4 Cronograma real	16
7. Desarrollo e implementación	17
7.1 Gestión de usuarios	17
7.2 Gestión de tareas	17

7.3 Generación automática del horario	18
7.4 Lugares y desplazamientos	18
7.5 Compartición y sincronización	19
7.6 Ajustes	20
7.7 Base de datos	21
7.8 Seguridad.....	22
8. Pruebas realizadas	23
8.1 Problemas encontrados y soluciones	24
9. Funcionalidades no implementadas o pendientes	24
9.1 Propuestas de mejora	25
10. Conclusiones	25
11. Bibliografía	26
12. Anexos	26

1. Introducción

Este proyecto consiste en el desarrollo de una aplicación móvil para Android diseñada para la gestión inteligente de tareas, agendas, ubicaciones y viajes. La aplicación está diseñada para ayudar a los usuarios a organizar automáticamente sus actividades diarias, considerando no solo la duración de cada tarea, sino también su prioridad, frecuencia, conexiones con otras tareas, ubicación y posibles desplazamientos entre ubicaciones.

La idea central del proyecto surge de la necesidad de una herramienta más flexible que un calendario tradicional. Muchas aplicaciones permiten a los usuarios crear eventos o recordatorios, pero generalmente requieren que programen manualmente cada tarea para una hora específica. Este proyecto propone un sistema en el que los usuarios pueden ingresar tareas con restricciones de tiempo, y la aplicación intenta generar automáticamente una agenda adecuada.

Además, el proyecto incluye un componente para el intercambio de datos entre dispositivos mediante el protocolo UDP. Esta funcionalidad está diseñada para permitir a los usuarios compartir una parte específica de su árbol de tareas con otros usuarios externos sin revelar toda su información personal. Para lograr esto, se distingue entre usuarios reales, usuarios que intercambian datos localmente y usuarios externos.

La aplicación se desarrolla utilizando Android Studio y utiliza una base de datos local para almacenar de forma persistente información sobre usuarios, tareas, lugares, viajes, permisos, solicitudes de acceso y datos necesarios para la sincronización.

1.2 Justificación del proyecto

La gestión del tiempo es un reto común en la vida diaria, especialmente al compaginar estudios, trabajo, desplazamientos, responsabilidades personales y tareas recurrentes. Los calendarios tradicionales permiten marcar eventos, pero no siempre ayudan a determinar cuándo completar una tarea si no tiene una hora fija.

Por este motivo, el proyecto ofrece una aplicación que no solo almacena tareas, sino que también permite visualizar las relaciones entre ellas. Una tarea puede depender de otra, tener subtareas, estar limitada a ciertos días, tener una duración específica o requerir estar en una ubicación determinada.

Otro aspecto importante del proyecto es la posibilidad de compartir solo un subconjunto de tareas. Por ejemplo, un usuario podría querer compartir únicamente las tareas relacionadas con un proyecto, estudio o actividad específicos con otra persona, sin revelar el resto de su agenda. Para solucionar este problema, el proyecto propone el uso de usuarios locales y recursos compartidos.

En la empresa había visto como se utilizan los UUID para evitar problemas de clave duplicados utilice el UUID de versión 7.

El proyecto también se justifica por su valor técnico, ya que integra distintos conocimientos del ciclo de Desarrollo de Aplicaciones Multiplataforma:

- Programación.
- Se utilizó lenguaje JAVA para programar todas las funcionalidades de la aplicación.
- Acceso a datos.
- El proyecto contiene una base de datos SQL a la que tengo que acceder
- También algunos campos en la base de datos están guardados en formato JSON.
- Desarrollo de interfaces.
- Para la aplicación se desarrolló interfaz propia utilizando XML.
- Programación de servicios y procesos.
- Algunas tareas se hacen en hilos aparte para que el usuario puede seguir utilizar la aplicación
- Programación multimedia y dispositivos móviles.
- Es una aplicación móvil
- Sistemas de gestión empresarial.
- La aplicación contiene gestión de tiempo que es esencial para gestión de trabajo.
- Lenguajes de marcas y sistemas de gestión de información.
- Para interfaz y algunos recursos se utiliza XML y JSON.
- Entornos de desarrollo.
- Para desarrollar la aplicación se utilizó Android Studio que es un entorno de desarrollo.

3. Objetivos

El objetivo general del proyecto es desarrollar una aplicación móvil Android para la gestión automática de tareas y horarios, permitiendo al usuario crear tareas jerárquicas,

asociarlas a lugares, calcular o registrar tiempos de desplazamiento y compartir su información con otros dispositivos.

3.1 Objetivos específicos

Objetivos específicos del proyecto:

- Desarrollar una base de datos local capaz de almacenar usuarios, tareas, historial de cambios, ubicaciones, viajes y permisos de acceso.
- Implementar un sistema de usuarios con diferentes tipos: usuario real, usuario local y usuario externo.
- Habilitar la creación, modificación, desactivación y organización jerárquica de tareas.
- Registrar los cambios en las tareas mediante una tabla de historial para restaurar el estado de la programación.
- Desarrollar un sistema para generar automáticamente programaciones basadas en las restricciones de cada tarea.
- Habilitar la gestión de ubicaciones y viajes entre ellas.
- Preparar una aplicación para calcular tiempos de viaje usando Google Maps cuando el usuario configure una clave API.
- Habilitar el uso compartido de un subárbol o del árbol de tareas completo mediante recursos compartidos.
- Desarrollar un sistema de comunicación entre dispositivos usando el protocolo UDP.
- Implementar medidas de seguridad, como el almacenamiento de contraseñas cifradas y futuras comunicaciones encriptadas.
- Crear una interfaz sencilla para iniciar sesión, gestionar tareas, consultar programaciones, gestionar ubicaciones, configurar parámetros y controlar usuarios externos.

4. Análisis de necesidades y requisitos

La necesidad principal detectada es la falta de una herramienta que permita crear tareas de forma flexible y que sea capaz de organizar automáticamente el horario del usuario. En una agenda tradicional, cada evento suele tener una fecha y hora concreta. Sin embargo, en la vida real muchas tareas no tienen una hora fija, sino un rango posible, una duración aproximada o una prioridad.

Por ejemplo, una persona puede necesitar estudiar durante dos horas esta semana, hacer una compra antes del viernes o realizar una tarea en algún momento libre del día. En estos casos, el usuario sabe qué debe hacer, pero no siempre sabe cuándo colocarlo en su horario. La aplicación intenta resolver este problema generando un horario automático en función de las restricciones.

También existe la necesidad de representar tareas compuestas. Por ejemplo, una tarea padre llamada “Instituto” puede contener subtareas como “Clase de programación”, “Proyecto” o “Estudiar”. Esto permite organizar la información en forma de árbol y facilita que las subtareas hereden ciertas restricciones de la tarea principal.

Otra necesidad importante es la compartición selectiva. El usuario puede querer compartir solo una parte de su agenda, no toda. Para ello se propone un sistema donde un usuario local representa un recurso compartido concreto, cuyo origen es una tarea raíz seleccionada por el usuario.

4.1 Problema detectado

Actualmente muchas personas tienen dificultades para organizar correctamente su tiempo. Aunque existen calendarios, agendas y aplicaciones de tareas, la mayoría de estas herramientas obligan al usuario a decidir manualmente cuándo debe realizar cada actividad. Esto puede ser un problema cuando el usuario tiene muchas tareas, poca disponibilidad, diferentes prioridades o actividades que no tienen una hora fija.

El problema principal detectado es que las aplicaciones tradicionales suelen funcionar como simples listas o calendarios, pero no ayudan de forma suficiente a tomar decisiones sobre la planificación diaria. El usuario debe pensar dónde colocar cada tarea, calcular si tiene tiempo suficiente, recordar los desplazamientos entre lugares y reorganizar manualmente el horario cuando aparece un cambio.

Además, muchas personas no solo necesitan guardar tareas, sino también estructurarlas. Por ejemplo, una actividad principal puede contener subtareas, puede tener una duración determinada, puede repetirse ciertos días o puede depender de un lugar concreto. Si esta información no se gestiona correctamente, es fácil olvidar tareas, planificar mal el día o crear horarios poco realistas.

Por ello, se detecta la necesidad de una aplicación que permita registrar tareas de forma flexible y que, además, ayude al usuario a generar un horario automático teniendo en cuenta restricciones como la duración, el rango horario, la periodicidad, la jerarquía entre tareas, los lugares y los desplazamientos.

4.2 Usuarios de la aplicación

La aplicación está dirigida a cualquier persona que necesite organizar mejor su tiempo y sus tareas diarias. No está pensada únicamente para un perfil profesional concreto, sino para usuarios que quieran planificar actividades personales, académicas o laborales de una forma más estructurada.

Uno de los perfiles principales son las personas que quieren organizar su vida cotidiana. Este tipo de usuario puede utilizar la aplicación para planificar estudios, trabajo, compras, citas, actividades personales, tareas del hogar o rutinas semanales.

También está pensada para personas con mala memoria o que suelen olvidar tareas importantes. La aplicación permite registrar las actividades y consultarlas posteriormente, reduciendo la posibilidad de olvidar compromisos o tareas pendientes.

Otro grupo de usuarios son las personas con dificultades para tomar decisiones. En muchas ocasiones, el problema no es saber qué tareas hay que hacer, sino decidir cuándo hacerlas. La generación automática del horario ayuda a reducir esta carga, ya que el sistema intenta colocar las tareas en los huecos disponibles según sus restricciones.

La aplicación también puede ser útil para personas con dificultades de organización. Al permitir crear tareas padre e hijas, establecer duraciones, lugares, días y acciones, el usuario puede estructurar mejor sus obligaciones y ver su planificación de forma más clara.

Por último, también puede ser utilizada por estudiantes, trabajadores, personas con rutinas complejas o usuarios que necesitan coordinar actividades en diferentes lugares, ya que la aplicación tiene en cuenta la gestión de lugares y desplazamientos.

4.3 Alcance del proyecto

Funcionalidades incluidas:

- Creación de tareas, grupos de tareas y su gestión.
- Posibilidad a Conectar dispositivos externos para obtener datos usando UDP.
- Tener en cuenta el tiempo de viaje.

Funcionalidades excluidas:

- Gestión de tareas grandes o proyectos en cuales no se sabe cuánto tiempo se va a gastar.
- Editar datos desde dispositivo externo.

4.5 Análisis de mercado

El mercado de aplicaciones de productividad y gestión del tiempo es altamente competitivo, con numerosas herramientas consolidadas como Todoist, Trello, Toggl Track, TMetric y Tiimo. Estas aplicaciones permiten a los usuarios gestionar tareas, organizar proyectos, controlar el tiempo y planificar sus actividades diarias. Por lo tanto, la competencia es intensa, lo que dificulta la diferenciación.

Esta aplicación destaca porque no solo permite a los usuarios registrar tareas, sino que también intenta programarlas automáticamente en función de restricciones, duración, ubicación y relaciones con otras tareas. Esto supone una clara ventaja sobre los calendarios o gestores de tareas tradicionales.

Desde una perspectiva técnica, las barreras de entrada son bajas, ya que el desarrollo de aplicaciones móviles es actualmente relativamente económico. Sin embargo, atraer usuarios, generar confianza y lograr el reconocimiento del mercado requiere un esfuerzo considerable. Además, existen numerosos productos alternativos, como agendas de papel, calendarios digitales, aplicaciones para tomar notas y herramientas de organización personal.

Desde una perspectiva social, existe una creciente demanda de herramientas que ayuden a los usuarios a gestionar mejor su tiempo, especialmente en el contexto del teletrabajo, los estudios y la necesidad de aumentar la productividad personal. Desde el punto de vista tecnológico, este sector ofrece oportunidades gracias a la integración con calendarios, mapas, notificaciones e inteligencia artificial.

4.6 Requisitos funcionales

RF01. La aplicación debe permitir crear usuarios.

RF02. La aplicación debe permitir iniciar sesión con un nombre de usuario existente.

RF03. El campo de nombre de usuario debe funcionar como campo de texto y como selector de usuarios ya creados.

RF04. Al crear un usuario real, la aplicación debe crear automáticamente una tarea raíz de tipo Empty.

RF05. La aplicación debe permitir crear tareas con nombre, tipo, duración, horario, estado, lugar y acciones asociadas.

RF06. La aplicación debe permitir organizar las tareas en forma de árbol mediante tareas padre e hijas.

RF07. La aplicación debe ocultar al usuario la tarea raíz automática, mostrando solo sus hijos.

RF08. La aplicación debe permitir crear tareas de tipo Empty para organizar otras tareas.

RF09. La aplicación debe permitir modificar tareas.

RF10. Cuando una tarea se modifica, debe conservarse la versión anterior para mantener un historial correcto.

RF11. La aplicación debe permitir desactivar tareas sin eliminarlas físicamente de la base de datos.

RF12. La aplicación debe registrar los cambios de tareas mediante TaskChange.

RF13. La aplicación debe generar un horario automático a partir de las tareas y sus cambios.

RF14. Las tareas con State igual a false no deben tenerse en cuenta en el horario.

RF15. Las tareas muteadas deben aparecer en el horario, pero no deben ejecutar sus acciones.

RF16. La aplicación debe permitir crear lugares.

RF17. La aplicación debe permitir crear viajes entre lugares.

RF18. Si no existe un viaje entre dos lugares necesarios para el horario, la aplicación debe crearlo y solicitar al usuario que complete los datos.

RF19. La aplicación debe permitir compartir una parte del árbol de tareas.

RF20. La aplicación debe permitir añadir usuarios externos.

RF21. La aplicación debe permitir bloquear usuarios externos no deseados.

RF22. La aplicación debe permitir configurar ajustes como petición de contraseña, API Key de Google Maps y tiempo adicional para viajes.

RF23. La pantalla principal debe permitir navegar por el horario mediante gestos.

RF24. La debe tener dos idiomas (Inglés y Castellano)

RF25. Las contraseñas deben almacenarse mediante hash.

RF26. La comunicación entre dispositivos debe prepararse para utilizar cifrado con clave pública.

RF27. La clave privada se plantea como derivada a partir de la contraseña del usuario.

RF28. Los usuarios externos no deben poder editar los datos compartidos.

RF29. Los usuarios externos solo deben poder consultar datos y en caso si estapermetido editar algunos campos locales.

RF30. Debe existir la posibilidad de bloquear usuarios externos.

RF31. El usuario debe poder activar la opción de pedir contraseña al entrar en su cuenta.

4.7 Requisitos no funcionales

RNF01. La aplicación debe funcionar en dispositivos Android.

RNF02. Los datos deben almacenarse de forma persistente en una base de datos local.

RNF03. La interfaz debe ser comprensible y sencilla para el usuario.

RNF04. El sistema debe mantener la coherencia entre la interfaz y la base de datos.

RNF05. La aplicación debe evitar la pérdida de información histórica.

RNF06. El diseño debe permitir ampliaciones futuras, como sincronización completa o integración con Google Maps.

RNF07. La aplicación debe intentar generar horarios de forma eficiente.

RNF08. La aplicación debe estar preparada para funcionar sin conexión a Internet, salvo las funciones que dependan de servicios externos.

5. Diseño de la solución software

Para el desarrollo del proyecto se ha utilizado Android Studio como entorno de desarrollo principal. La aplicación está planteada como una aplicación móvil nativa para Android.

El lenguaje principal utilizado en el proyecto es Java, junto con archivos XML para la definición de interfaces, recursos y configuración de la aplicación. La persistencia de datos se realiza mediante una base de datos local compatible con Room/SQLite.

También se utiliza Gradle como sistema de construcción del proyecto. La estructura del proyecto incluye el archivo AndroidManifest, clases Java, recursos XML y el archivo build.gradle.kts.

En el diseño de la base de datos se han definido distintas entidades para usuarios, tareas, cambios, lugares, viajes, recursos compartidos, permisos de acceso, solicitudes, sincronización y mensajes pendientes.

5.2 Arquitectura general

La aplicación se organiza en varias capas lógicas:

- Capa de interfaz: Incluye pantallas de inicio de sesión, página principal, creación de tareas, gestión de usuarios externos, ubicaciones, viajes y configuración.
- Capa de lógica: Se encarga de aplicar las reglas del sistema, como la creación automática de una tarea raíz, la validación de tareas secundarias, la generación de horarios y la monitorización del estado.
- Capa de almacenamiento de datos: Se encarga de almacenar y recuperar datos mediante una base de datos local.
- Capa de comunicación: Diseñada para enviar y recibir datos entre dispositivos mediante el protocolo UDP.
- Capa de seguridad: Se encarga de la protección mediante contraseña, el control de acceso y el cifrado de las comunicaciones.

5.3 Modelo de datos

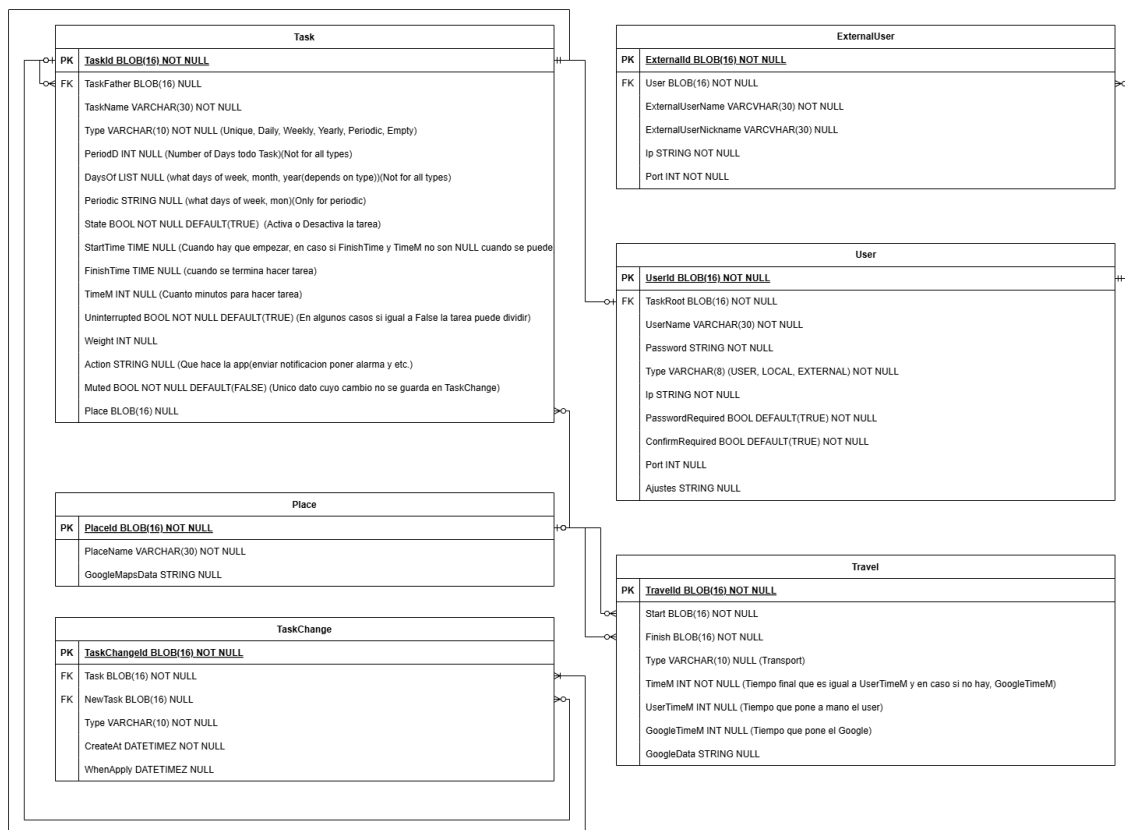
El modelo de datos se ha diseñado alrededor de la entidad Task, ya que las tareas son el elemento principal del sistema. Cada tarea puede tener una tarea padre, lo que permite construir una estructura jerárquica en forma de árbol.

La tabla User almacena la información de los usuarios. Cada usuario tiene un identificador, un nombre único, una contraseña, un tipo, dirección IP, puerto y ajustes. Además, cada usuario real tiene asociada una tarea raíz.

La tabla Task almacena las tareas. Incluye campos como identificador, tarea padre, nombre, tipo, periodo, días, estado, hora de inicio, hora de fin, duración, prioridad, acciones, estado de silencio y lugar.

La tabla TaskChange almacena el historial de cambios de las tareas. Esta tabla es fundamental porque permite saber cuándo una tarea fue creada, modificada, desactivada o sustituida por otra. Además, se utiliza como base para la generación automática del horario.

Las tablas Place y Travel permiten gestionar lugares y desplazamientos. Place almacena los lugares y Travel almacena el tiempo necesario para ir de un lugar a otro.

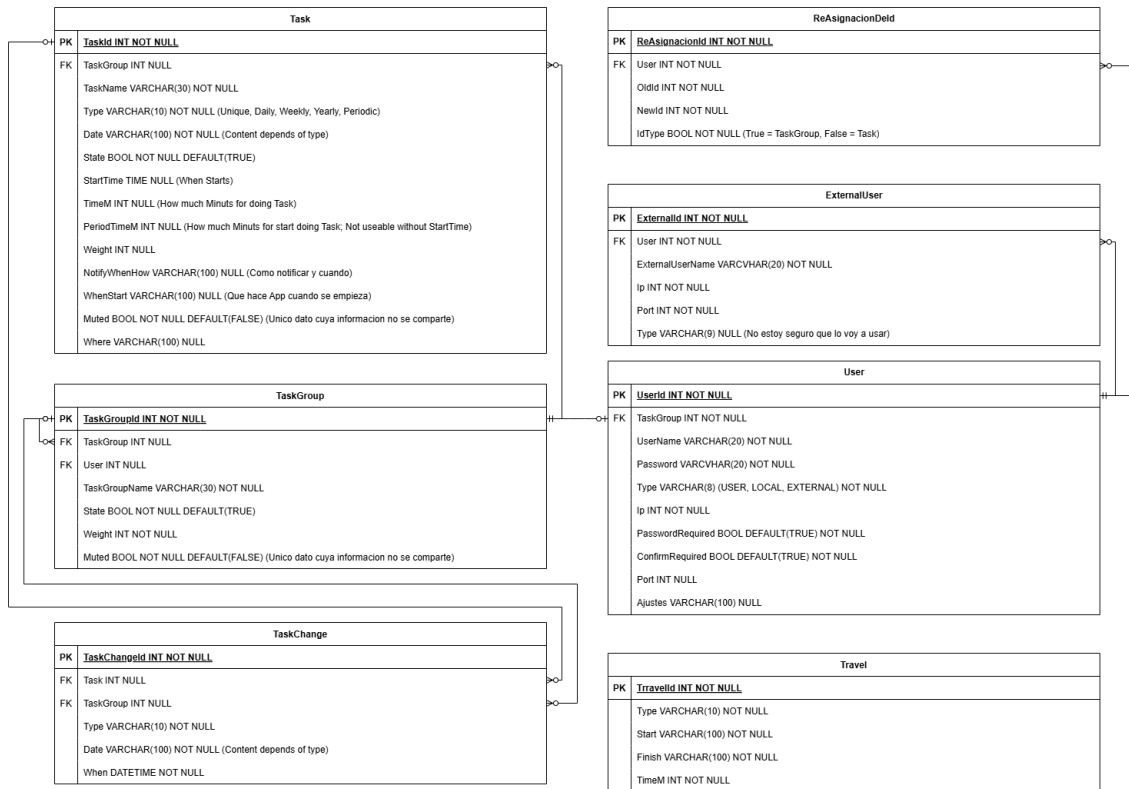


Esquema previa de base de datos V2

Pero esto no es primera versión antes de eso tenía otro modelo cual fue utilizando mayoría de tiempo.

En ese modelo no tenía tabla Place, pero no es muy importante, porque había que implementar la sin dudas.

La diferencia principal es la tabla TaskGroup, que es en teoría Tarea de tipo Empty, pero ese modelo tenía ciertas limitaciones ya que en ese modelo tareas no podían ser padres. Ejemplo tengo escuela desde 8:30 hasta 14:15 y dentro de ella tengo clase de Programación 8:30-9:20 en primera versión no fue posible en la segunda es posible.



Esqema previa de base de datos V2

5.4 Nombre y logo

El nombre (ThothTask) directamente junto el elemento distintivo de la aplicación (Thot) que es el dios egipcio asociado con el conocimiento, la escritura, el tiempo y la sabiduría y la palabra que representa mejor la aplicación (Tarea) ya que es una aplicación de gestión de tareas y traduce las a inglés.

El logotipo de la aplicación consta de tres elementos principales: un calendario, un reloj y la imagen de Thot.

El calendario simboliza la planificación de tareas, fechas, días y periodos. Está directamente relacionado con la función principal de la aplicación, ya que los usuarios organizan sus actividades en un horario y pueden visualizar su plan por día, semana, mes o año.

El reloj simboliza la gestión del tiempo, la duración de las tareas y la importancia de distribuir adecuadamente las actividades a lo largo del día. La aplicación utiliza el tiempo no solo para indicar las horas de inicio y finalización, sino también para calcular la duración de las tareas, los intervalos de tiempo disponibles y los desplazamientos entre ubicaciones.

La imagen de Thot se utiliza como elemento distintivo y simbólico. En la mitología egipcia, Thot se asocia con la sabiduría, la escritura, los cálculos y el orden. Por lo tanto, su presencia en el logotipo refleja el aspecto intelectual de la aplicación: no se trata simplemente de registrar tareas, sino de ayudar al usuario a organizar su horario de forma lógica y automática.

La combinación de estos elementos transmite la idea central del proyecto: una aplicación que combina planificación, gestión del tiempo y organización inteligente. Además, el uso de un personaje mitológico distingue visualmente la aplicación de otras herramientas de productividad más generales.

5.5 Diseño de la interfaz y Flujos

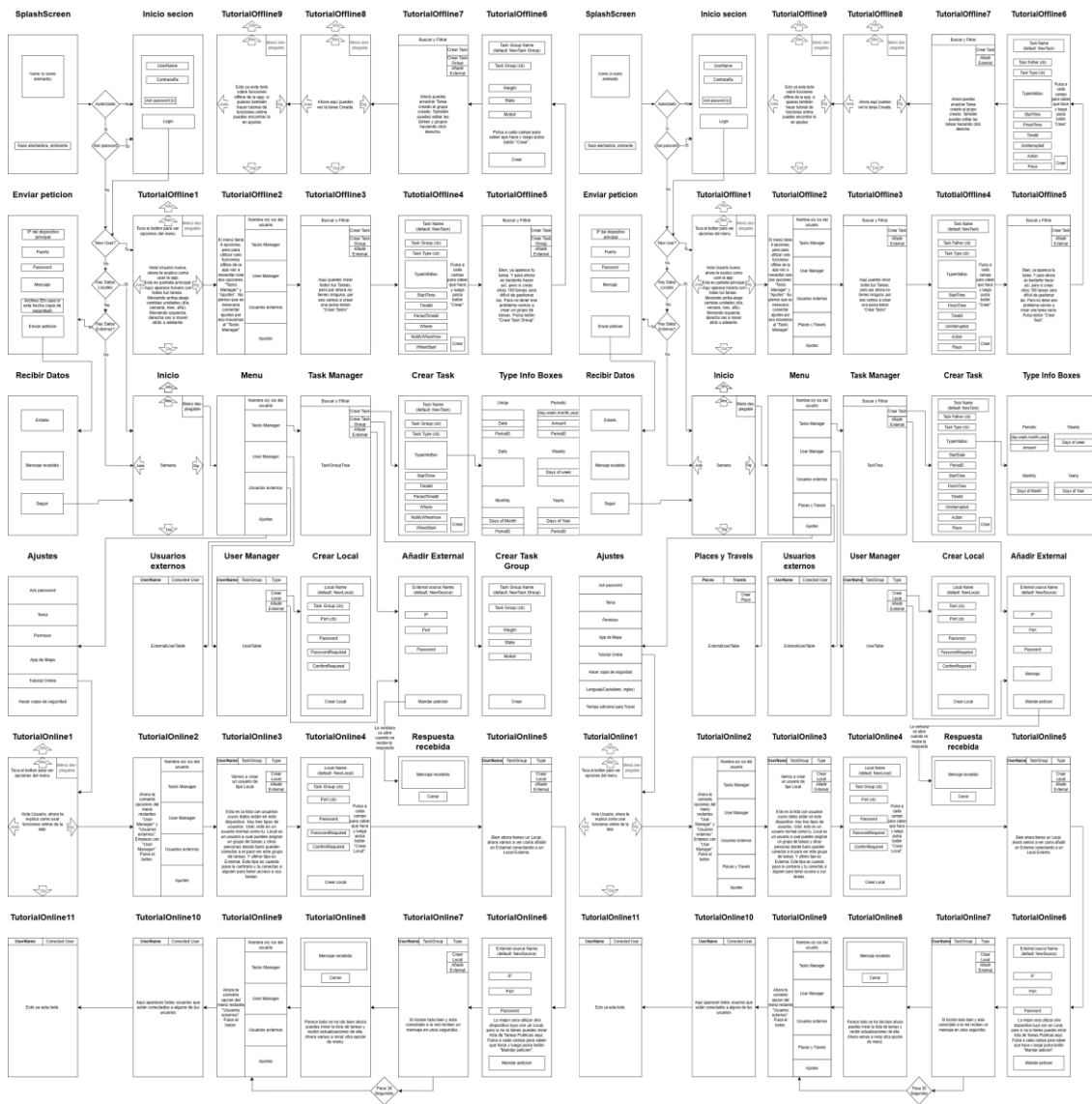
Ya que para mí interfaz es solo el medio de comunicación entre usuario y el código. El objetivo principal fue que todas funcionalidades fueron accesibles, por eso en general tenía una pantalla para cada tabla de base de datos. Para TaskChange eso fue pantalla principal ya que el horario se genera a partir él. Task, User, ExternalUser, Places, Travels tenían sus propios managers.

Y por eso cuando hice cambio de esquema de base de datos también cambié esquema de interfaz.

Pero como se puede ver en esquemas hay más pantallas:

- Inicio de sesión
- Ajustes
- Menú

También se puede ver que quería implementar más funcionalidades como tutorial, recuperación de los datos y opciones de ajustes adicionales.



Esquemas de diseño previo

6. Planificación del proyecto

La realidad fue muy diferente de expectativas.

6.1 Metodología empleada

Se ha optado por una metodología híbrida combinando un enfoque tradicional en la fase inicial del proyecto para establecer una base sólida y documentada del sistema, seguida de una metodología ágil basada en Scrum para la evolución incremental del producto mediante Sprints de desarrollo.

Esta combinación permite garantizar entrega de una base estable y, a la vez, flexibilidad para adaptar funcionalidades.

6.2 Fases

Fase inicial - enfoque tradicional (Waterfall controlado): análisis de requisitos y diseño arquitectónico claro para establecer la base.

Fase de desarrollo e iteración - Scrum: una vez disponible el MVP, trabajar en Sprint cortos (1–2 semanas) para añadir integraciones, optimizaciones y mejoras.

Fase final – Pruebas finales, documentación y entrega.

6.3 Cronograma pensada

En teoría empecé trabajar en el proyecto 19 de diciembre, pero por los estudios y exámenes no había avanzado. Empecé hacer proyecto en modo intenso en 16 de febrero. Termine con Fase inicial en 2 de marzo. En ese día tenía terminado Plan de Trabajo y Análisis con diseño previo (en total más o menos fueron 75h). No lo hice antes por que en principio no sabía cómo debe estar la aplicación, pero después de esa fase ya tenía más o menos todo claro. Después viene Fase de desarrollo e iteración (175h) y al final Fase final (20h).

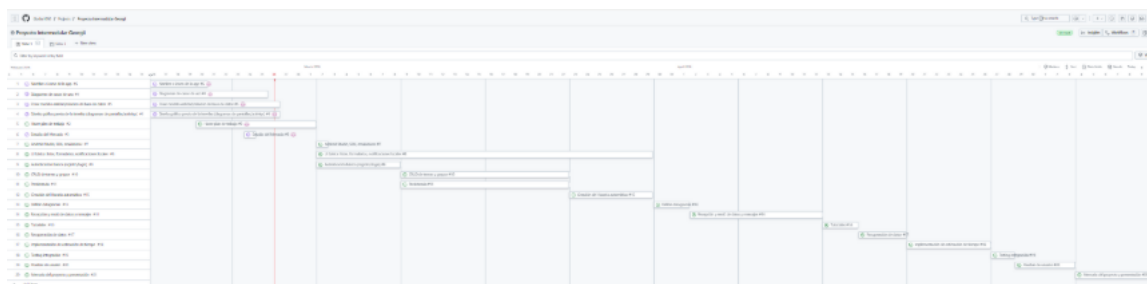


Diagrama de Gantt de cronograma pensada

6.4 Cronograma real

El problema más grande que tenía es el cambio de modelo de datos en 15 de abril, por lo cual casi todo que tenía hecho antes de eso perdió sentido y por eso tenía que rehacer todo, casi desde 0. Y por eso perdió muchas cosas (no lo puso en diagrama de Gantt ya que no está dentro del proyecto final, pero al final la versión antes del cambio tenía más o menos lo mismo, pero sin casi nada de UDP, aunque tenía calculo automático de desplazamiento, pero sin probar ya que API de Google a requiere tarjeta para obtener api key). Y por eso me movió las fechas muchísimo y quite algunas cosas que quería hacer, pero no tenía tiempo.

Fase de desarrollo e iteración antes de cambio de modelo (75h más o menos)

Fase de desarrollo e iteración antes de cambio de modelo (105h más o menos)

Fase final (10h más o menos).

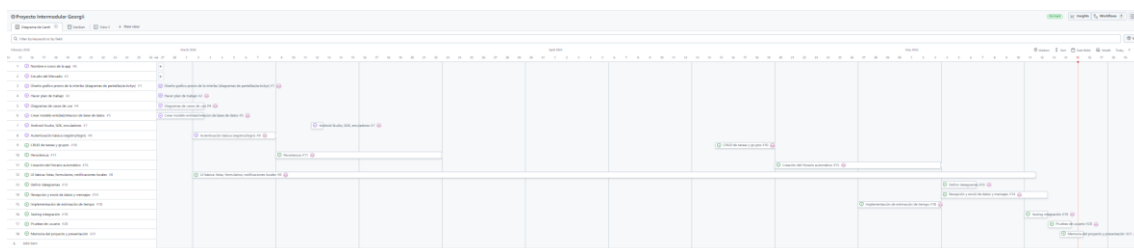


Diagrama de Gantt de cronograma real

7. Desarrollo e implementación

7.1 Gestión de usuarios

La aplicación distingue entre tres tipos principales de usuario:

- Usuario: representa al usuario real de la aplicación.
- Local: representa a un usuario local utilizado para compartir únicamente un conjunto específico de tareas.
- Externo: representa una fuente de datos externa obtenida de otro dispositivo.

Al crear un usuario real, el sistema genera automáticamente una tarea raíz de tipo "Empty". Esta tarea no es visible directamente para el usuario y sirve como tarea principal común para todas las tareas que el usuario cree sin seleccionar una tarea principal específica.

El nombre de usuario debe ser único. En la pantalla de inicio de sesión, el campo "UserName" está diseñado para funcionar simultáneamente como campo de texto y selector de usuarios existentes. Esto permite al usuario iniciar sesión en una cuenta existente o introducir un nuevo nombre para crear una nueva cuenta.

7.2 Gestión de tareas

La gestión de tareas es la funcionalidad principal de la aplicación. Cada tarea puede contener información temporal, como la hora de inicio, la hora de finalización, la duración, los días de recurrencia o un período. También puede tener una ubicación asociada y una acción configurada.

Las tareas se pueden organizar jerárquicamente. Una tarea secundaria forma parte de una tarea principal y, por lo tanto, debe ajustarse a sus restricciones horarias. Por ejemplo, si una tarea principal es el horario de una clase de 8:00 a 14:00, sus subtareas deben estar dentro de ese horario.

Las tareas vacías tienen una función especial. Se utilizan para crear tareas raíz o para organizar otras tareas, aunque el usuario también puede crearlas para estructurar su planificación.

Eliminar una tarea no la borra físicamente si forma parte del historial. En su lugar, se le asigna un estado de false y un State de Mute es true. Esto conserva la información necesaria para restaurar correctamente el historial.

7.3 Generación automática del horario

El cronograma no se almacena como una tabla fija, sino que se genera automáticamente en función de las tareas y el historial almacenado en la tabla TaskChange.

La tabla TaskChange muestra cuándo se creó, modificó, desactivó o reemplazó una tarea. Con esta información, el sistema puede reconstruir qué tareas estaban activas en un momento dado y generar un cronograma coherente.

Las tareas flotantes son aquellas que tienen restricciones, pero no tienen una hora asignada específica. Para estas tareas, la aplicación debe encontrar espacios disponibles y asignarles horas específicas. Por defecto, se generan refinamientos para el mes siguiente. Si el usuario solicita una fecha posterior, el sistema puede generar temporalmente nuevos refinamientos.

Cuando se modifica, crea o elimina una tarea, los refinamientos futuros que aún no se hayan utilizado deben eliminarse y regenerarse, ya que el cronograma puede haber cambiado.

7.4 Lugares y desplazamientos

La aplicación incluye un sistema de gestión de ubicación y viajes. Esta funcionalidad es importante porque el horario del usuario depende no solo de la duración de las tareas, sino también de su ubicación y del tiempo necesario para desplazarse entre ellas.

El almacenamiento de datos en la base de datos se realiza mediante la entidad "Place". Cada lugar representa una ubicación que puede asociarse a una tarea. Por ejemplo, una tarea podría completarse en casa, en la escuela, en el trabajo o en cualquier otra ubicación definida por el usuario.

Los viajes se representan mediante la entidad "Travel". Un evento de viaje vincula el punto de partida con el destino y almacena el tiempo necesario para completarlo. Este tiempo puede provenir de diversas fuentes: puede ser introducido manualmente por el usuario, calculado posteriormente con Google Maps o ajustado para tener en cuenta el tiempo adicional especificado en la configuración.

El objetivo de esta función es hacer que la generación automática de horarios sea más realista. Si un usuario tiene dos tareas consecutivas en ubicaciones diferentes, la aplicación debe tener en cuenta el tiempo de viaje necesario. Si falta un registro de viaje entre dos ubicaciones requeridas, el sistema puede crear un registro de viaje pendiente y solicitar al usuario que complete la información.

Además, el sistema permite que las tareas secundarias hereden dinámicamente la ubicación de la tarea principal. Esto elimina la necesidad de modificar todas las subtareas cuando cambia la ubicación de la tarea principal. De esta forma, el sistema admite una estructura más flexible y reduce la duplicación de datos.

Si bien la integración automática con Google Maps sigue siendo un aspecto a mejorar, el sistema ya almacena datos relacionados con mapas, tiempos estimados y tiempos especificados por el usuario.

7.5 Compartición y sincronización

La aplicación está diseñada para permitir el intercambio selectivo de información entre dispositivos. La idea principal es que los usuarios no tengan que compartir todo su calendario, sino solo una parte específica de su árbol de tareas, aunque pueden hacerlo si lo desean.

Para ello, se distinguen varios tipos de usuario. El tipo "User" representa al usuario real de la aplicación. El tipo "Local" se utiliza para compartir una parte específica de las tareas, normalmente un subárbol seleccionado por el usuario. El tipo "Externo" representa una fuente de datos externa obtenida de otro dispositivo.

El intercambio se basa en recursos compartidos. Un recurso compartido puede representar una tarea raíz específica y todas sus subtareas. De este modo, si un usuario desea compartir solo las tareas relacionadas con un proyecto, lección o actividad específicos, puede hacerlo sin revelar el resto de su información personal.

Los usuarios externos pueden solicitar acceso a los recursos compartidos. Dependiendo de la configuración, el acceso puede requerir una contraseña o la confirmación del propietario. El sistema también incluye la posibilidad de bloquear a usuarios externos no deseados.

La sincronización se implementa mediante comunicación UDP. Este sistema permite enviar y recibir mensajes entre dispositivos mediante una dirección IP y un puerto. La base de datos incluye entidades para gestionar solicitudes, permisos, mensajes pendientes, mensajes recibidos y el estado de sincronización.

En cuanto a los permisos, los usuarios externos no deben poder modificar directamente los datos originales del propietario. Su acceso se limita a la visualización de la

información compartida. En algunos casos, pueden realizar cambios locales, como desactivar las notificaciones, pero estos cambios no afectan a los datos originales.

Actualmente, el componente de intercambio y sincronización está desarrollado en gran medida a nivel de diseño y estructura, pero aún no es completamente estable. La comunicación UDP, el manejo de errores, la confirmación de mensajes y el cifrado seguro aún deben finalizarse.

7.6 Ajustes

La aplicación incluye una pantalla de configuración que permite al usuario modificar diversos ajustes relacionados con el comportamiento general de la app, alarmas, permisos, programación automática, navegación e idioma.

Esta pantalla carga la configuración del usuario actual desde la base de datos. Estos ajustes se almacenan en formato JSON en el campo de usuario correspondiente. Este método permite guardar varios ajustes personalizables sin necesidad de crear una columna independiente para cada uno.

Una de las secciones principales de la pantalla de configuración está relacionada con las alarmas. El usuario puede activar o desactivar los sonidos de alarma y decidir si desea que el dispositivo vibre. Estos ajustes permiten al usuario personalizar el comportamiento de las notificaciones según sus preferencias.

La pantalla también permite al usuario revisar y solicitar los permisos necesarios. Por un lado, el usuario puede solicitar el permiso de notificaciones, necesario en las versiones recientes de Android para mostrar alertas. Por otro lado, puede acceder a la configuración del sistema para otorgar acceso al modo No molestar. Este permiso es útil si la aplicación necesita un control más preciso sobre alertas o alarmas importantes.

Otra opción importante es "Requerir contraseña". Si está habilitada, la aplicación debe solicitar una contraseña al iniciar sesión en la cuenta del usuario. Si está deshabilitada, el acceso puede ser más rápido si existe una sesión previa. Esta opción permite equilibrar la comodidad y la seguridad según las necesidades del usuario.

También puede configurar el número de días de antelación con los que la aplicación debe programar actividades. Este valor se utiliza para generar acciones o alertas futuras. Si el usuario no introduce un valor válido, la aplicación utiliza el valor predeterminado de 60 días.

Para los viajes, se pueden definir dos parámetros de tiempo adicionales en la pantalla de configuración: tiempo obligatorio y tiempo extra. El tiempo obligatorio se define como los minutos necesarios antes o después de un viaje, por ejemplo, para prepararse, cambiarse de ropa o llegar con el tiempo mínimo. El tiempo extra se puede usar como

margen de seguridad sugerido para evitar prisas. Estos valores se pueden sumar al tiempo de viaje para crear un horario más realista.

En la pantalla de configuración, también puede cambiar el idioma de la aplicación entre español e inglés. Esto se logra modificando la configuración regional de la aplicación, lo que permite que la interfaz se adapte al idioma seleccionado por el usuario.

Cuando el usuario guarda los cambios, la aplicación actualiza el archivo JSON de configuración en la base de datos. Tras guardar, se reprograma el período configurado para que la nueva configuración se aplique inmediatamente a la aplicación.

7.7 Base de datos

La base de datos está formada por varias entidades relacionadas.

Las tablas principales de la aplicación:

- User almacena usuarios. Sus campos principales son UserId, TaskRoot, UserName, Password, Type, Ip, Port, PasswordRequired, ConfirmRequired y Ajustes.
- Task almacena las tareas. Sus campos principales son TaskId, TaskFather, TaskName, Type, PeriodD, DaysOf, Periodic, State, StartTime, FinishTime, TimeM, Uninterrupted, Weight, Action, Muted y Place.
- TaskChange almacena el historial de cambios de tareas. Sus campos principales son TaskChangeld, Task, NewTask, Type, CreateAt y WhenApply.

Tablas de desplazamiento:

- Place almacena lugares. Sus campos principales son PlaceId, PlaceName y GoogleMapsData.
- Travel almacena desplazamientos entre lugares. Sus campos principales son TravelId, Start, Finish, Type, TimeM, UserTimeM, GoogleTimeM y GoogleData.

Las tablas que están relacionadas con la compartición de datos y el control de acceso de usuarios externos:

- ExternalUser almacena usuarios externos que pueden acceder a datos compartidos. Incluye ExternalId, User, ExternalUserName, ExternalUserNickname, Ip, Port y Blocked.
- ShareResource representa recursos compartidos. Incluye ResourceId, OwnerUserId, Type, Name, RootTaskId, Port, PasswordRequired, ConfirmRequired y Active.

- AccessGrant almacena permisos concedidos. Incluye GrantId, ExternalUserId, ResourceId, Granted, GrantedAtUtcMs y RevokedAtUtcMs.
- AccessRequest almacena solicitudes de acceso. Incluye información sobre el usuario externo, el recurso solicitado, el estado de la solicitud y datos de red.

Las tablas que están relacionadas con la sincronización, recepción de datos externos y modificaciones locales sobre tareas importadas:

- ExternalSource representa fuentes externas de datos. Permite almacenar información de recursos remotos, clave pública, estado de bloqueo y si se incluyen o no en el horario.
- SyncState almacena el estado de sincronización con otros dispositivos.
- PendingOutbox almacena mensajes pendientes de enviar.
- ReceivedInbox almacena mensajes recibidos para evitar procesarlos más de una vez.
- TaskOverlay permite aplicar cambios locales sobre tareas externas, como silenciar una tarea o modificar su acción local.



Esquema de base de datos final

7.8 Seguridad

La seguridad del proyecto se considera desde varias perspectivas.

En primer lugar, las contraseñas de los usuarios no deben almacenarse en texto plano, sino como hashes. Esto reduce el riesgo de acceso no autorizado a la base de datos.

En segundo lugar, el intercambio de datos entre dispositivos mediante UDP debe estar protegido. La solución propuesta consiste en utilizar cifrado de clave pública para evitar que terceros puedan leer fácilmente los mensajes.

También se propone generar o derivar la clave privada del usuario a partir de su contraseña. Esto permite vincular la seguridad del sistema con la autenticación del usuario.

En cuanto a los permisos, los usuarios externos no pueden editar directamente los datos del propietario. Su acceso se limita a la visualización de la información compartida. En algunos casos, pueden bloquear acciones localmente, pero no modificar la información original.

8. Pruebas realizadas

Durante el desarrollo se realizaron pruebas funcionales manuales para comprobar el comportamiento de las principales pantallas y operaciones de la aplicación.

Las pruebas realizadas fueron:

- Creación de usuario nuevo.
- Inicio de sesión con usuario existente.
- Comprobación de unicidad del nombre de usuario.
- Creación automática de tarea raíz.
- Creación de tareas normales y tareas Empty.
- Asignación de tarea padre.
- Modificación de tareas.
- Desactivación de tareas sin eliminación física.
- Registro de cambios en TaskChange.
- Creación de lugares.
- Creación de viajes entre lugares.
- Visualización de tareas en la pantalla principal.
- Navegación básica por la pantalla de inicio.

- Configuración de ajustes.

8.1 Problemas encontrados y soluciones

Durante el desarrollo del proyecto surgieron varios desafíos técnicos y de diseño.

Uno de los principales problemas fue la complejidad del modelo de tareas. Una tarea es más que un simple evento con fecha y hora; puede tener restricciones, subtareas, duración, frecuencia, ubicación, estado y acciones asociadas. Para abordar esto, se desarrolló una tabla de tareas con múltiples campos y relaciones jerárquicas utilizando TaskFather.

Otro problema fue la necesidad de mantener el historial. Si una tarea se elimina físicamente, el sistema pierde la información necesaria para restaurar programaciones anteriores. Por lo tanto, se decidió que las tareas en uso no se eliminan, sino que se desactivan.

La generación automática de programaciones también presentó desafíos. Para abordarlo, se propuso TaskChange como fuente de información de historial confiable y como base para determinar qué tareas deben programarse en un momento dado.

El intercambio de datos a través de UDP también presentó un desafío, ya que requiere la gestión de mensajes, reintentos, recepciones duplicadas, sincronización y seguridad. Por lo tanto, se desarrollaron tablas como PendingOutbox, ReceivedInbox y SyncState.

Finalmente, la integración con Google Maps se dejó como una mejora o una función prevista, ya que requiere una clave API, una conexión externa y el procesamiento de las respuestas de la API.

9. Funcionalidades no implementadas o pendientes

A pesar de contar con una sólida base de diseño e implementación, algunas funcionalidades aún no están completamente implementadas.

La programación avanzada de tareas aún no está totalmente implementada. La idea es que algunas tareas tengan una ubicación de programación fija, mientras que otras se programen automáticamente por el sistema.

El intercambio de datos a través de UDP está bastante desarrollado en cuanto a estructura y diseño, pero aún no es completamente estable. Los aspectos de recepción, validación, sincronización y manejo de errores todavía requieren mejoras.

El cálculo automático del tiempo de viaje mediante la API de Google Maps tampoco está completamente implementado. El sistema actualmente está listo para almacenar datos

de Google y permite al usuario configurar una clave API, pero la integración completa con un servicio externo aún está en desarrollo.

El cifrado completo de datos a través de UDP también está pendiente de mejora. El proyecto incluye el uso de una clave pública y la generación de clave privada a partir de una contraseña, pero requiere una implementación y pruebas más exhaustivas.

9.1 Propuestas de mejora

Como futuras mejoras del proyecto se proponen las siguientes:

- Finalizar el algoritmo de generación automática de horario.
- Implementar completamente la fijación de tareas.
- Mejorar la comunicación UDP entre dispositivos.
- Integrar completamente la API de Google Maps para calcular tiempos de desplazamiento.
- Mejorar la interfaz de selección de tarea padre mediante un árbol visual.
- Añadir copia de seguridad y restauración de datos.
- Implementar exportación e importación de datos.
- Añadir pruebas unitarias y pruebas automáticas.
- Mejorar la accesibilidad de la interfaz.
- Añadir estadísticas de uso del tiempo.
- Permitir sincronización en segundo plano.

10. Conclusiones

El proyecto dio como resultado el desarrollo de una aplicación Android con un enfoque vanguardista para la gestión de tareas y la generación automática de horarios. A diferencia de un calendario tradicional, la aplicación no se limita a almacenar eventos; busca reflejar restricciones del mundo real como la duración, la frecuencia, la jerarquía, la ubicación y los desplazamientos.

Un aspecto crucial del proyecto es el diseño de la base de datos, ya que permite el historial de cambios, el intercambio de partes individuales de las tareas y la preparación para la sincronización entre dispositivos. La tabla TaskChange desempeña un papel

fundamental, proporcionando un historial preciso y sirviendo como base para la generación de horarios.

Si bien algunas funcionalidades aún no están completamente desarrolladas, como la comunicación UDP segura y el cálculo automático mediante Google Maps, el proyecto ha sentado una base técnica sólida y escalable. Además, permitió aplicar conocimientos especializados en programación, bases de datos, diseño de interfaces.

Finalmente, el proyecto presentó un desafío significativo debido a la complejidad de las reglas de y la cantidad de entidades relacionadas. Sin embargo, también brindó la oportunidad de desarrollar una solución cercana al problema real e integrar gran parte del conocimiento adquirido durante el ciclo. Por el número de problemas que aparecieron durante el desarrollo estoy bastante contento con resultado.

11. Bibliografía

Google. Android Developers. "Save data in a local database using Room". Disponible en: <https://developer.android.com/training/data-storage/room/>

Google. Android Developers. "Material Components". Disponible en: <https://developer.android.com/design/ui/mobile/guides/components/material-overview>

Google. Google Maps Platform Documentation. "Routes API". Disponible en: <https://developers.google.com/maps/documentation/routes>

Copilot para consultas (Ej. "Como se crea cifrado UDP mediante claves", "Que opciones tengo para trabajar con bases de datos en Android Studio")

12. Anexos

Repositorio GitHub con Proyecto:

<https://github.com/Gosha1030/ProyectoIntermodularGeorgii>